EmoSense — Android App Documentation

Kotlin + Jetpack Compose app: recorder, on-device login, API client, wellness messages

Contents of this document

- 04-android/README.md

Generated from the Markdown files in the docs folder (the Markdown files remain the source of truth).

4. The Android app (Kotlin + Jetpack Compose)

Source: `android/` — package `com.example.emotion`, files in `android/app/src/main/java/com/example/emotion/`. Installable files: `apk/EmoSense-release.apk` (about 1 MB) and `apk/EmoSense-debug.apk`.

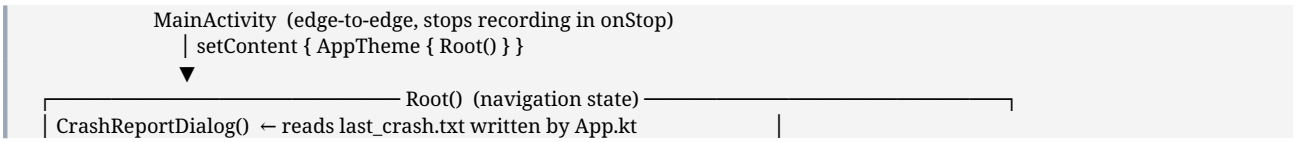
Purpose

The user-facing part of the system: **sign in → record the voice → send it to the API → show the detected emotion with a wellness message.** It must be light, simple and robust on Android 16.

Why this component

Piece	What	Why chosen	Rejected alternative
Kotlin + Jetpack Compose	Modern native Android UI	Little code, current standard, no XML layouts	XML views (verbose), Flutter/React Native (extra tooling, larger apps)
`AudioRecord`	Records raw 16 kHz mono PCM	We need exactly the format the model expects; no conversion on the server	`MediaRecorder` (compressed formats like AAC → decoding + quality loss)
OkHttp	HTTP client	Reliable timeouts, multipart upload, the Android standard	`HttpURLConnection` (verbose), Retrofit (extra library for one endpoint)
`org.json` (built into Android)	Reads the JSON answer	Zero extra dependencies → smaller APK	Gson/Moshi (more size for little benefit)
`LocalAuth` (on-device accounts)	User ID + password stored on the phone	Needs no database/server code; the free server stays stateless	Server-side accounts (need a persistent database)
Settings screen	Server address, test button	The URL can change (free hosts, new deployments) without rebuilding the app; hidden from the login screen	Hard-coded URL only
Crash reporter	Saves a crash stack trace and shows it next launch	No developer tools were available on the phone	Guessing why the app closed
Wellness content in `Theme.kt`	Headline, insight and 3 tips per emotion	The project is a health & wellness application: the result must be useful, not just a label	Showing only the emotion name
R8 shrinking in the release build	Removes unused code/resources	APK of ≈ 1 MB instead of 9 MB	Debug build only
`targetSdk 36`, edge-to-edge handling	Android 16 behaviour	Compatibility with the user's phone	Targeting an old SDK

Architecture diagram



```
┌──────────────────────────────────────────────────────────────────────┐
│ Login ──▶ Register        Settings (server URL + Test connection)    │
│   │       │                ▲          ▲                              │
│   └── LocalAuth (SharedPreferences, salted hash)  ⚙ from Login and Main │
│          ▼                                                           │
│        Main ── WavRecorder ──▶ WAV bytes ──▶ EmotionApi.predict ──▶ Prediction │
│          └── ResultCard: emotion, confidence, wellness tips (Theme.kt), bars │
└──────────────────────────────────────────────────────────────────────┘
```

File map

File	Responsibility
`App.kt`	`Application` class: crash reporter (`CrashReport`)
`MainActivity.kt`	Entry point, `Root()` navigation, constants (`DEFAULT_BASE_URL`, `MAX_RECORD_SECONDS`)
`Screens.kt`	Login, Register, Settings, Main screens, result card, crash dialog
`LocalAuth.kt`	Register/login/sign-out with salted, iterated SHA-256 hashes
`WavRecorder.kt`	Microphone → WAV bytes
`EmotionApi.kt`	`health()` and `predict()` calls, JSON parsing
`Theme.kt`	Colours, per-emotion wellness text, disclaimer notes
`AndroidManifest.xml`, `build.gradle.kts`	Permissions, app class, SDK targets, shrinking

Line-by-line walkthrough

1. `WavRecorder.kt` — the microphone

```
object WavRecorder {
    const val SAMPLE_RATE = 16_000
    private val mainHandler = Handler(Looper.getMainLooper())
```

`object` = a single shared instance (no need to create one). 16 000 Hz matches the model. `mainHandler` posts small tasks to the **main (UI) thread**: Android UI may only be touched from it.

```
    suspend fun record(
        maxSeconds: Int,
        stop: AtomicBoolean,
        minSeconds: Int = 1,
        onSeconds: (Float) -> Unit = {},
    ): ByteArray = withContext(Dispatchers.IO) {
```

A **suspend function** (a Kotlin coroutine that can pause without blocking the UI). `withContext(Dispatchers.IO)` runs the body on a background thread — recording must never block the screen. `stop` is a thread-safe flag the button sets; `onSeconds` reports progress; the result is the finished WAV file as bytes.

```
        val minBuf = AudioRecord.getMinBufferSize(SAMPLE_RATE, AudioFormat.CHANNEL_IN_MONO,
AudioFormat.ENCODING_PCM_16BIT)
        if (minBuf <= 0) throw IllegalStateException("This device does not support 16 kHz recording (buffer size $minBuf)")
```

Ask Android for the smallest internal buffer for 16 kHz mono 16-bit audio. A value ≤ 0 means the device cannot record in this format → a readable error instead of a crash.

```
        val recorder = AudioRecord(
            MediaRecorder.AudioSource.MIC, SAMPLE_RATE, AudioFormat.CHANNEL_IN_MONO,
            AudioFormat.ENCODING_PCM_16BIT, minBuf * 4,
```

```
        )
```

Create the recorder: source = microphone, 16 kHz, mono, 16-bit samples, buffer four times the minimum (prevents dropped audio if the app is briefly slow).

```
            if (recorder.state != AudioRecord.STATE_INITIALIZED) {
                throw IllegalStateException("Microphone could not be opened (is another app using it?)")
            }
```

Check the recorder really opened (another app may hold the microphone, or permission may be missing).

```
            val total = SAMPLE_RATE * maxSeconds
            val pcm = ShortArray(total)
            var read = 0
            var lastReported = -1f
            recorder.startRecording()
```

Allocate room for up to 15 s (`16 000 × 15 = 240 000` 16-bit samples ≈ 0.5 MB) and start.

```
            while (read < total && !(stop.get() && read >= SAMPLE_RATE * minSeconds)) {
                val n = recorder.read(pcm, read, minOf(2048, total - read))
                if (n < 0) throw IllegalStateException("Microphone read failed (error $n)")
                read += n
```

Loop until the buffer is full **or** the user asked to stop (but record at least 1 s). Each `read` fetches up to 2048 samples (128 ms) from the microphone; a negative number is an Android error code.

```
                val seconds = read.toFloat() / SAMPLE_RATE
                if (seconds - lastReported >= 0.25f) {
                    lastReported = seconds
                    mainHandler.post { onSeconds(seconds) }
                }
```

Convert samples to seconds and notify the UI **at most four times per second**, always through the main thread. (An earlier version updated UI state from the audio thread, which is one of the things the redesign removed.)

```
            toWav(pcm.copyOf(read))
        } finally {
            try { recorder.stop() } catch (_: Throwable) { }
            recorder.release()
        }
```

Cut the array to the number of samples actually recorded and wrap it into a WAV. The `finally` block **always** releases the microphone, even after an error — otherwise other apps could not use it.

```
    private fun toWav(pcm: ShortArray): ByteArray {
        val dataLen = pcm.size * 2
        val bb = ByteBuffer.allocate(44 + dataLen).order(ByteOrder.LITTLE_ENDIAN)
        bb.put("RIFF".toByteArray()).putInt(36 + dataLen).put("WAVE".toByteArray())
        bb.put("fmt ".toByteArray()).putInt(16).putShort(1).putShort(1)             // PCM, mono
        bb.putInt(SAMPLE_RATE).putInt(SAMPLE_RATE * 2).putShort(2).putShort(16)     // rate, byte rate, align, bits
        bb.put("data".toByteArray()).putInt(dataLen)
        pcm.forEach { bb.putShort(it) }
        return bb.array()
    }
```

A WAV file is a **44-byte header** + **raw samples**. The header says: `RIFF` container, total size, `WAVE`; the `fmt ` block (PCM format 1, 1 channel, sample rate, bytes per second, bytes per sample,

16 bits); the `data` block with its length. Everything is **little-endian** (least significant byte first) as the WAV format requires. Then each 16-bit sample is appended.

2. `EmotionApi.kt` — talking to the server

Two small data classes carry the answer: `Prediction` (emotion, confidence, probabilities, `speechDetected`, latency, windows) and `WindowResult` (start time, emotion and confidence of one 3-second window).

```
private val client = OkHttpClient.Builder()
    .connectTimeout(30, TimeUnit.SECONDS)
    .readTimeout(90, TimeUnit.SECONDS)
    .writeTimeout(30, TimeUnit.SECONDS)
    .build()
```

One reusable HTTP client. **A 90-second read timeout** lets the app wait for the free server to wake up and for a slow CPU to finish the analysis; the default (10 s) would fail on cold starts.

```
private fun url(path: String) = baseUrl.trim().trimEnd('/') + path
```

Joins the base address and a path safely (removes spaces and a trailing slash typed in Settings).

```
suspend fun health(): Long? = withContext(Dispatchers.IO) {
    try {
        val req = Request.Builder().url(url("/health")).get().build()
        val t0 = System.nanoTime()
        client.newCall(req).execute().use { r -> if (r.isSuccessful) (System.nanoTime() - t0) / 1_000_000 else null }
    } catch (e: Exception) {
        null
    }
}
```

Calls `GET /health`, measures the round trip in milliseconds, and returns it — or `null` if the server cannot be reached. `use { }` closes the response automatically. The Settings "Test connection" button and the "Server online" text use it.

```
suspend fun predict(wav: ByteArray, retries: Int = 2): Prediction = withContext(Dispatchers.IO) {
    val body = MultipartBody.Builder().setType(MultipartBody.FORM)
        .addFormDataPart("file", "recording.wav", wav.toRequestBody("audio/wav".toMediaType()))
        .build()
    val req = Request.Builder().url(url("/predict")).post(body).build()
```

Build a **multipart/form-data** upload with one field named `file` — exactly what the FastAPI parameter `file: UploadFile = File(...)` expects — and a `POST` request to `/predict`.

```
    var last: IOException? = null
    for (attempt in 0..retries) {
        try {
            client.newCall(req).execute().use { r ->
                val text = r.body?.string().orEmpty()
                if (!r.isSuccessful) throw IOException("Server error ${r.code}: ${text.take(160)}")
                return@withContext parse(JSONObject(text))
            }
        } catch (e: IOException) {
            last = e
            if (attempt < retries) delay(2_000L * (attempt + 1))   // back off, then retry (cold start)
        }
    }
    throw last ?: IOException("request failed")
```

Try up to **3 times** (the first plus 2 retries). A non-2xx status becomes an error with a short piece of the server's message; on success parse the JSON and return immediately

(return@withContext). Between attempts wait 2 s, then 4 s (**back-off**), which helps when a sleeping server is starting.

```
    private fun parse(j: JSONObject): Prediction {
        val probs = j.optJSONObject("probabilities")
        val list = ArrayList<Pair<String, Double>>()
```

Turn the JSON into Kotlin objects. opt… functions return a default instead of throwing when a field is missing — safer than get….

```
            list.sortByDescending { it.second }
```

Sort the (emotion, probability) pairs so the most likely emotion comes first in the bars.

3. LocalAuth.kt — accounts on the phone

```
class LocalAuth(context: Context) {
    private val prefs = context.applicationContext.getSharedPreferences("auth", Context.MODE_PRIVATE)
```

SharedPreferences = a small private key-value file belonging to this app only. applicationContext avoids leaking an Activity.

```
    fun register(userId: String, password: String, confirm: String): AuthResult {
        val id = userId.trim()
        if (!ID_RE.matches(id)) return AuthResult(false, "User ID must be 3–20 characters: letters, digits, dot, dash or underscore.")
        if (password.length < MIN_PASSWORD) return AuthResult(false, "Password must be at least $MIN_PASSWORD characters.")
        if (password != confirm) return AuthResult(false, "The two passwords do not match.")
        if (prefs.contains(hashKey(id))) return AuthResult(false, "This User ID already exists on this phone. Please sign in.")
```

Validation first: allowed characters/length (regular expression), minimum password length, matching confirmation, and uniqueness (case-insensitive because hashKey lower-cases).

```
        val salt = ByteArray(16).also { SecureRandom().nextBytes(it) }
        prefs.edit()
            .putString(hashKey(id), toHex(salt) + ":" + toHex(hash(password, salt)))
            .putString(nameKey(id), id)
            .putString(KEY_SESSION, id)
            .apply()
        return AuthResult(true)
```

Create a **random 16-byte salt** with a cryptographic random generator; store salt:hash (never the password) plus the display name, and sign the user in. apply() writes in the background.

```
    private fun hash(password: String, salt: ByteArray): ByteArray {
        val md = MessageDigest.getInstance("SHA-256")
        md.update(salt)
        var d = md.digest(password.toByteArray(Charsets.UTF_8))
        repeat(ITERATIONS) {
            md.update(d)
            md.update(salt)
            d = md.digest()
        }
        return d
    }
```

A **salted, iterated hash**. SHA-256 is a one-way function. The **salt** makes two users with the same password get different hashes (and defeats pre-computed tables); repeating it **20 000 times** makes each password guess slow for an attacker who copies the file, while still taking only a few milliseconds for a legitimate login. (Real systems use PBKDF2/bcrypt/Argon2; this simple scheme works on all Android versions the app supports.)

```
        return MessageDigest.isEqual(expected, hash(password, salt))      // constant-time comparison
```

`isEqual` compares **in constant time**, so the time taken does not reveal how many leading bytes matched.

```
        val lockedUntil = prefs.getLong(KEY_LOCKED_UNTIL, 0L)
        if (now < lockedUntil) return AuthResult(false, "Too many attempts. Try again in ${(lockedUntil - now + 999) / 1000} s.")
```

After 5 wrong passwords the login is **locked for 30 seconds** (the counter and the unlock time are stored, so restarting the app does not reset them).

```
    private fun verify(password: String, stored: String): Boolean {
        val parts = stored.split(":")
        if (parts.size != 2) return false
        val salt = fromHex(parts[0]) ?: return false
        val expected = fromHex(parts[1]) ?: return false
        return MessageDigest.isEqual(expected, hash(password, salt))      // constant-time comparison
    }
```

`verify` re-creates the hash from the typed password and the stored salt, and compares it with the stored hash. `fromHex` converts the stored hexadecimal text back into bytes (`null` if the text is damaged, so a corrupted entry simply fails the login instead of crashing).

```
    fun signOut() {
        prefs.edit().remove(KEY_SESSION).apply()
    }
```

Sign-out only forgets who is signed in; accounts remain on the phone.

Honest limits: accounts exist only on this phone (uninstalling or clearing data deletes them; there is no password recovery) and they protect the *app screen*, not the cloud API.

4. `Theme.kt` — the wellness content

```
data class EmotionStyle(
    val color: Color,
    val emoji: String,
    val title: String,
    val headline: String,       // one-line reading of the voice
    val insight: String,        // short, kind explanation
    val tips: List<String>,     // three small, doable actions
)
```

Everything the UI needs for one emotion in one object: accent colour, emoji, name, a headline, a gentle explanation and three practical tips.

```
const val LOW_CONFIDENCE_NOTE = "I am not very sure about this reading. Try recording again, a little closer to the microphone, for a few seconds."
const val SADNESS_NOTE = "If this feeling stays heavy for more than two weeks, consider talking to a doctor or counsellor."
const val SAFETY_NOTE = "If you ever feel unsafe or in crisis, contact your local emergency services or a trusted person right away."
const val DISCLAIMER = "This is an AI estimate from your voice, not a medical diagnosis."
```

Safety wording that appears with the results: a low-confidence hint, a gentle referral line for sadness, a general emergency note (no country-specific numbers) and the disclaimer. **Why:** emotion recognition is probabilistic and the topic is health; the app must not pretend to diagnose.

```
fun styleOf(emotion: String?): EmotionStyle = STYLES[emotion] ?: DefaultStyle
```

Looks up the style for the label returned by the API (`"sad"`, …) and falls back to a neutral "Ready" style for unknown/`null` values.

5. `MainActivity.kt` — entry point and navigation

```
    fun openSettings() { settingsReturn = screen; screen = Screen.Settings }
```

Remembers which screen opened Settings (`settingsReturn`) and then shows it, so Back/Save return to Login *or* Main correctly.

```
const val DEFAULT_BASE_URL = "https://ser-api-r6mj.onrender.com"
const val MAX_RECORD_SECONDS = 15
val stopRecording = AtomicBoolean(false)
```

Default server; the longest recording (the server scores up to five 3-second windows); a thread-safe flag used to stop recording from the button or when the app leaves the screen.

```
        enableEdgeToEdge()      // Android 15/16 draw apps edge-to-edge; the root below adds the system-bar insets
```

Android 15+ draws apps **behind** the status and navigation bars. We opt in explicitly and add padding below so nothing is hidden.

```
    override fun onStop() {
        stopRecording.set(true)
        super.onStop()
    }
```

When the user leaves the app, a running recording stops cleanly (the microphone is only allowed while the app is visible).

```
private enum class Screen { Login, Register, Main, Settings }
```

The four screens; the app keeps a single variable saying which one is shown (simple state-based navigation — no navigation library needed, which keeps the app small).

```
    var user by remember { mutableStateOf(auth.currentUser()) }
    var screen by remember { mutableStateOf(if (user != null) Screen.Main else Screen.Login) }
```

`remember { mutableStateOf(...) }` is Compose's **observable state**: when the value changes, the screen redraws. If a user is already signed in the app opens straight on the main screen.

```
    BackHandler(enabled = screen == Screen.Settings || screen == Screen.Register) {
        screen = if (screen == Screen.Settings) settingsReturn else Screen.Login
    }
```

Handles the system **Back gesture** (Android 14+ predictive back): from Settings return where we came from, from Register return to Login.

```
    Box(Modifier.fillMaxSize().safeDrawingPadding(), contentAlignment = Alignment.TopCenter) {
        Box(Modifier.widthIn(max = 600.dp).fillMaxSize()) {
```

`safeDrawingPadding()` keeps content clear of the status bar, navigation bar, camera cut-out and keyboard; `widthIn(max = 600.dp)` keeps the layout readable on tablets and foldables.

6. `Screens.kt` — the screens

`Screens.kt` contains one composable per screen: `LoginScreen`, `RegisterScreen`, `SettingsScreen` and `MainScreen`, plus small helpers `InfoCard` (a message card used for errors and "no speech") and `ResultCard` (the result). A **composable** is a Kotlin function marked `@Composable` that describes a piece of UI; when the state it reads changes, Compose redraws only what changed.

Login — two text fields and a button:

```
            OutlinedTextField(
```

```
            password, { password = it }, label = { Text("Password") }, singleLine = true,
            visualTransformation = PasswordVisualTransformation(),
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Password),
            modifier = Modifier.fillMaxWidth(),
        )
```

`PasswordVisualTransformation` shows dots instead of characters; `KeyboardType.Password` tells the keyboard not to suggest or learn the text. **The server address does not appear here** — it is in Settings.

```
            onClick = {
                val result = auth.login(userId, password)
                if (result.ok) onSignedIn() else error = result.message
            },
```

Check the account locally; on success navigate to Main, otherwise show the message (wrong password, no such user, locked).

Settings — validates and tests the server address:

```
fun valid(): Boolean = (cleaned().startsWith("http://") || cleaned().startsWith("https://")) && cleaned().toHttpUrlOrNull() != null
```

Accept only real web addresses (OkHttp's parser rejects malformed ones).

```
                val ms = try { EmotionApi(cleaned()).health() } catch (e: CancellationException) { throw e } catch (e: Throwable) { null }
                status = if (ms != null) "✅ Connected (${ms} ms)" else "Could not reach the server. Check the address or try again in a
minute."
```

"Test connection" calls `/health`; a coroutine cancellation is re-thrown (never swallowed), every other failure becomes "could not reach".

Main screen — the recorder:

```
fun onButtonClicked() {
    val s = state
    if (s is UiState.Recording) {
        stopRecording.set(true)
    } else if (s !is UiState.Analysing) {
        if (ContextCompat.checkSelfPermission(ctx, Manifest.permission.RECORD_AUDIO) ==
PackageManager.PERMISSION_GRANTED) startRecording()
        else permissionLauncher.launch(Manifest.permission.RECORD_AUDIO)
    }
}
```

`onButtonClicked` is the **single handler of the big round button**: while recording it means *stop*; while analysing it is ignored; otherwise it starts recording — asking for the microphone permission first if it has not been granted.

```
fun startRecording() {
    if (state is UiState.Recording || state is UiState.Analysing) return
    scope.launch {
        try {
            stopRecording.set(false)
            state = UiState.Recording(0f)
            val wav = WavRecorder.record(maxSeconds = MAX_RECORD_SECONDS, stop = stopRecording) { sec ->
                if (state is UiState.Recording) state = UiState.Recording(sec)   // ignore late callbacks
            }
            state = UiState.Analysing
            state = UiState.Done(EmotionApi(baseUrl).predict(wav))
```

The **state machine**: `Idle → Recording → Analysing → Done/Error`. A guard line ignores taps while busy; `scope.launch` starts a coroutine; the recorder's callback updates the timer; after recording we switch to *Analysing* and call the API; the result becomes `Done`.

```
        } catch (e: CancellationException) {
            throw e
        } catch (e: Throwable) {
            Log.e(TAG, "recording/analysis failed", e)
            state = UiState.Error(friendlyError(e))
        }
```

Every failure is caught and shown as a message (and logged), so **no error can close the app**; the only exception re-thrown is coroutine cancellation, which must propagate.

```
    val permissionLauncher = rememberLauncherForActivityResult(ActivityResultContracts.RequestPermission()) { granted ->
        if (granted) startRecording()
        else state = UiState.Error("The microphone permission is needed to hear your voice. You can allow it in Settings ▸ Apps ▸
EmoSense ▸ Permissions.")
    }
```

The microphone is a **runtime permission**: Android shows a dialog the first time. We start recording only after the user taps *Allow*; if refused, a clear message explains how to fix it.

Result card — what the user sees:

```
        if (p.confidence < 0.5) {
            Text(LOW_CONFIDENCE_NOTE, fontSize = 13.sp, color = Color(0xFFB26A00), textAlign = TextAlign.Center)
        }
```

When the model is unsure (< 50 %), say so honestly and suggest recording again.

```
            st.tips.forEach { tip -> Text("•  $tip", fontSize = 14.sp, lineHeight = 20.sp) }
            if (emotion == "sad") Text(SADNESS_NOTE, fontSize = 13.sp, lineHeight = 18.sp, fontWeight = FontWeight.Medium)
```

The three wellness tips for the detected emotion, plus the extra referral line when the emotion is *sad*.

```
    val timeline = p.windows.mapNotNull { it.emotion }
    val changed = timeline.distinct().size > 1
```

For longer recordings, collect each window's emotion; if more than one different emotion appears, show "Your tone changed during the recording: 😐 → 😢". This uses the per-window results the API returns.

7. `App.kt` — the crash reporter

```
    Thread.setDefaultUncaughtExceptionHandler { thread, error ->
        try {
            File(filesDir, CrashReport.FILE).writeText(
                "Time: ${Date()}\n" +
                    "Device: ${Build.MANUFACTURER} ${Build.MODEL}, Android ${Build.VERSION.RELEASE} (API $
{Build.VERSION.SDK_INT})\n" +
                    "Thread: ${thread.name}\n\n" + Log.getStackTraceString(error)
            )
        } catch (_: Throwable) {
            // never let the reporter itself crash
        }
        previous?.uncaughtException(thread, error)
    }
```

If the app ever crashes, this handler **writes the error (with time, device and Android version) to a file** and then lets Android continue its normal crash handling. On the next launch `CrashReportDialog` shows the report with a *Copy* button. **Why:** the app was tested on a phone without a debugger; this turns an unexplained "app closed" into a precise error message.

8. Manifest and Gradle

```
    <uses-permission android:name="android.permission.INTERNET" />
```

```
    <uses-permission android:name="android.permission.RECORD_AUDIO" />
```

Only two permissions: network access and the microphone. No storage, location or contacts.

```
        android:name=".App"
```

Makes Android create our `App` class first, which installs the crash reporter.

```
        android:enableOnBackInvokedCallback="true"
```

Opts in to the Android 13+ **predictive back** system, matching the `BackHandler` usage.

```
    buildTypes {
        release {
            isMinifyEnabled = true          // R8 shrinks code ...
            isShrinkResources = true        // ... and resources -> much smaller APK
```

R8 removes code and resources that are never used and shortens names. It reduced the APK from about 9.3 MB (debug) to about 1.2 MB (release).

```
            signingConfig = signingConfigs.getByName("debug")   // demo build: signed with the debug key so it installs directly
```

Android refuses unsigned apps; for a college demo we sign the release build with the automatically created debug key so it installs without extra steps. (A store release would use a private key.)

```
        targetSdk = 36
```

Declares that the app is built for **Android 16 behaviour** (edge-to-edge enforcement, predictive back). `minSdk = 24` lets it run on older phones too.

Things to tell the mentor

- **Robustness over decoration:** every failure path shows a message; there is a crash reporter; recording is released in `finally`; UI state is updated from the right thread.
- **Wellness focus:** the output is a *supportive message*, with confidence-aware wording and a disclaimer — not a diagnosis.
- **Privacy:** audio is held in memory, sent over HTTPS to be analysed, and never stored on the phone or the server; passwords are stored only as salted hashes.
- **Light by design:** no navigation library, no JSON library, no image library — the release APK is about 1 MB.